11. Method References in Java — Quick Notes

Method References

Purpose

Shorter version of Lambda Expressions.

Used when Lambda: only calls another method

Syntax: ClassName::methodName
Or
object::methodName

Added in: Java 8

Lambda vs Method Reference

Lambda: x -> System.out.println(x)
Method Reference: System.out::println

Both do same work.

Basic Example

```
List<Integer> list = Arrays.asList(10,20,15,16);
list.forEach(System.out::println);
```

Output: 10 \n 20 \n 15 \n 16

When to Use?

Use method reference when Lambda:

- takes parameter(s)
- directly passes them to another method
- no extra logic

Static Method Reference

Example

```
class Gfg {
  static void printSquare(int x) {
     System.out.println(x * x);
  }
}
```

Usage: list.forEach(Gfg::printSquare);

Output: 100 \n 400 \n 225 \n 256

Types of Method References

Type	Syntax
Static Method	`ClassName::method`
Instance Method	`object::method`
Constructor	`ClassName::new`

Constructor Reference

Example: ArrayList::new

Uses: constructor reference

Comparator Example
Problem

Compare strings ignoring case.

String a[] = {"GFG", "IDE"};
String b[] = {"gfg", "ide"};

Method Reference with Comparator
Arrays.equals(a,b,String::compareToIgnoreCase);

Output:true

Equivalent Lambda
(x1, x2) -> x1.compareToIgnoreCase(x2)

Can be replaced by: String::compareToIgnoreCase

Why It Works?

Because: Comparator is a: Functional Interface

Method reference becomes implementation of: compare() method.

Important Quick Points

Topic	Point
Method Reference	Short form of Lambda
Symbol	`::`
Works with	Functional Interfaces
Static Method	`Class::method`
Object Method	`object::method`
Constructor	`Class::new`
Use Case	Cleaner readable code

Easy Memory Trick

Lambda	Method Reference
`x -> println(x)`	`System.out::println`
`(a,b) -> compare(a,b)`	`Class::compare`